Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras

Machine: Dell G15-5510 laptop, Intel Core i5-10200H, Ubuntu Linux (x86_64)
Compiler: clang, `-O3 -march=native`

Q1. Your Machine

Specification

Item	Value
CPU model	Intel Core i5-10200H @ 2.40 GHz
Microarchitecture	Comet Lake-H (10th Gen, Skylake-derived)
Physical cores	4 (all performance cores)
Logical cores	8 (2 SMT threads per core)
Performance / efficiency cores	4 P / 0 E (hybrid designs begin with 12th Gen)
Base clock	2.40 GHz
Max single-core clock	4.10 GHz
All-core clock under load	≈4.05 GHz (measured 4.03–4.09 GHz via `lscpu --extended`)
SIMD instruction set / width	AVX2 + FMA, 256-bit (8 fp32 lanes)
FMA units per core	2 (ports 0 and 5)
Vector registers per core	16 × 256-bit YMM
L1 data cache	32 KiB per core, 8-way (128 KiB total)
L2 cache	256 KiB per core, private (1 MiB total)
L3 cache	8 MiB, shared by all 4 cores, 16-way
Cache line size	64 bytes
RAM	16 GB (2×8 GiB Micron 4ATF1G64HZ-3G2E2 SODIMM)
RAM type / speed / channels	DDR4, modules rated 3200 MT/s, **configured at 2933 MT/s** (Comet Lake-H controller limit), dual-channel

Table 1: Machine specification. Verified with `lscpu`, `lscpu --caches`, `free -h` and `dmidecode -t memory`.

Derived quantities

Peak FP32 GFLOP/s, one core. Each AVX2 register holds 8 fp32 lanes; each of the 2 FMA units performs a fused multiply-add (2 flops) per cycle:

$$8 \text{ lanes} \times 2 \text{ FMA units} \times 2 \text{ flops} = 32 \text{ flops/cycle}$$

$$32 \times 4.05 \text{ GHz} = 129.6 \text{ GFLOP/s per core}$$

Peak FP32 GFLOP/s, all cores:

$$4 \times 129.6 = 518.4 \text{ GFLOP/s (theoretical)}$$

On a 45 W laptop part, power and thermal limits make the sustainable figure much lower.

Memory bandwidth. Using the *configured* DDR4-2933 speed (not the modules' rated 3200 MT/s, which the platform cannot run):

$$2 \text{ channels} \times 8 \text{ bytes} \times 2933 \text{ MT/s} = 46.9 \text{ GB/s}$$

Ridge points:

$$\text{one core: } \frac{129.6}{46.9} \approx 2.8 \text{ FLOP/byte} \qquad \text{all cores: } \frac{518.4}{46.9} \approx 11.1 \text{ FLOP/byte}$$

Q2. Naive Baseline

Input	Time (ms)	GFLOP/s	% of single-core peak
N=1024, d=64	124.7	2.15	1.7%
N=2048, d=64	583.2	1.84	1.4%
N=4096, d=128	5712.1	1.50	1.2%
N=4096, d=64	2757.0	1.56	1.2%

Table 2: Naive baseline, median of 5 runs, single thread.

Scaling with N and d. Doubling N from 2048 to 4096 (d=64) multiplies the time by ∼4.7×: the $O(N^2d)$ cost of $QK^\top$ and SV grows 4×, and the extra N^2 memory traffic (softmax and score matrix) no longer fits in cache, adding more. Doubling d at fixed N=4096 approximately doubles the time, matching the $O(N^2d)$ FLOP count.

Where the time goes. Step 3 of the naive code walks V column-wise with a d·4-byte stride, so every access opens a new cache line. Step 1's inner k-loop has a scalar accumulator, preventing safe vectorisation. The kernel delivers only ∼1.2% of the FMA peak, confirming it is starved by memory latency and poor locality rather than compute.

Q3. Optimisations

All variants pass `make check` (error $\leq 1.5 \times 10^{-6}$).

Variant	N=1024,d=64	N=2048,d=64	N=4096,d=128	N=4096,d=64
naive	124.7 (2.15)	583.2 (1.84)	5712.1 (1.50)	2757.0 (1.56)
loops	154.0 (1.74)	578.4 (1.86)	4595.0 (1.87)	2354.5 (1.82)
tiled	52.6 (5.10)	210.1 (5.11)	1953.9 (4.40)	860.6 (4.99)
fused	119.7 (2.24)	597.7 (1.80)	5300.4 (1.62)	2733.7 (1.57)
best	19.2 (14.00)	77.2 (13.90)	531.1 (16.17)	333.9 (12.86)

Table 3: Single-threaded times in ms (GFLOP/s in parentheses).

At N=4096, d=64 the speedups over naive are: loops 1.17×, tiled 3.20×, fused 1.01×, best 8.26×, omp (8 threads) 26.6×.

(a) attn_loops: loop reordering

Step 1 is reordered from *i*-*j*-*k* to *i*-*k*-*j* with S pre-zeroed: $Q[i][k]$ is hoisted out of the inner loop, K rows and the S row are accessed with stride 1, and the compiler auto-vectorises the j-loop. Step 3 is reordered from *i*-*k*-*j* to *i*-*j*-*k* so V and O are accessed contiguously. The gain

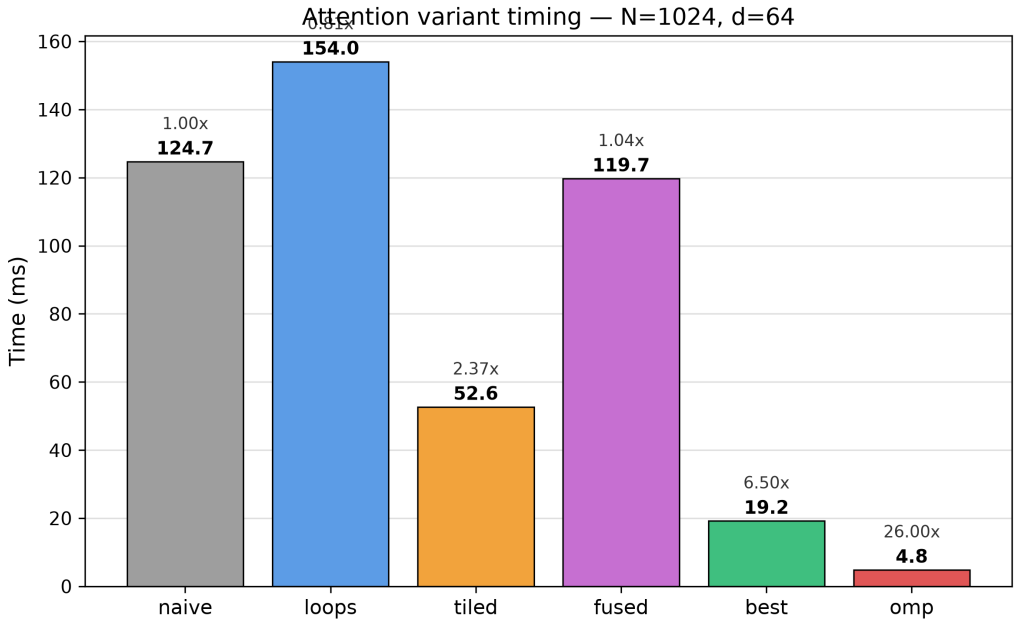


Figure 1: Variant timing, N=1024, d=64.

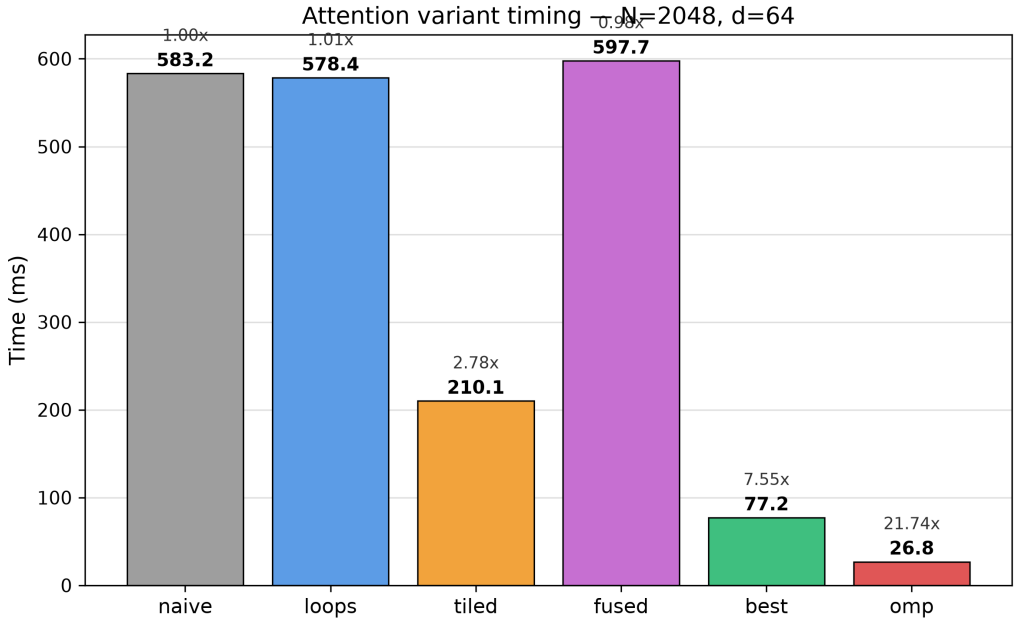


Figure 2: Variant timing, N=2048, d=64.

is modest (1.17×): the layout still forces the whole N^2 score matrix through memory, and the auto-vectoriser remains conservative. The limit afterwards is DRAM traffic on S, not instruction issue.

(b) attn_tiled: cache and register tiling

All three steps are blocked with tile sizes $T_I = T_J = 64$. A 64×64 tile of S (16 KiB) together with its Q and K slices fits in L1/L2, so each byte loaded from lower levels is reused across the whole tile before eviction. This yields 3.20× over naive. The remaining limit is that the N^2 matrix is still written once and read twice (softmax and SV); at ∼46.9 GB/s that compulsory

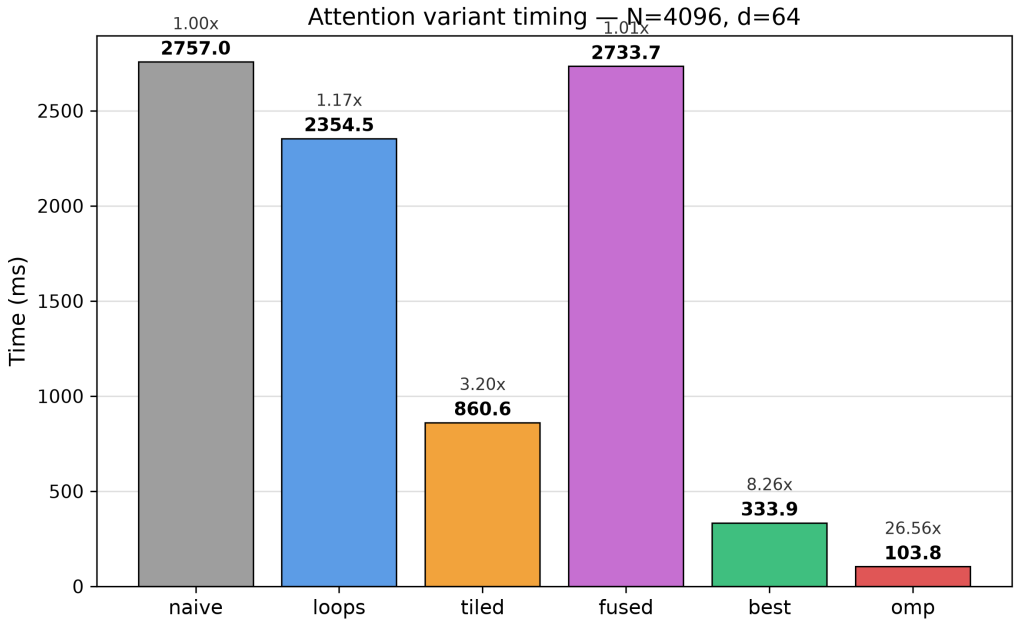


Figure 3: Variant timing, N=4096, d=64.

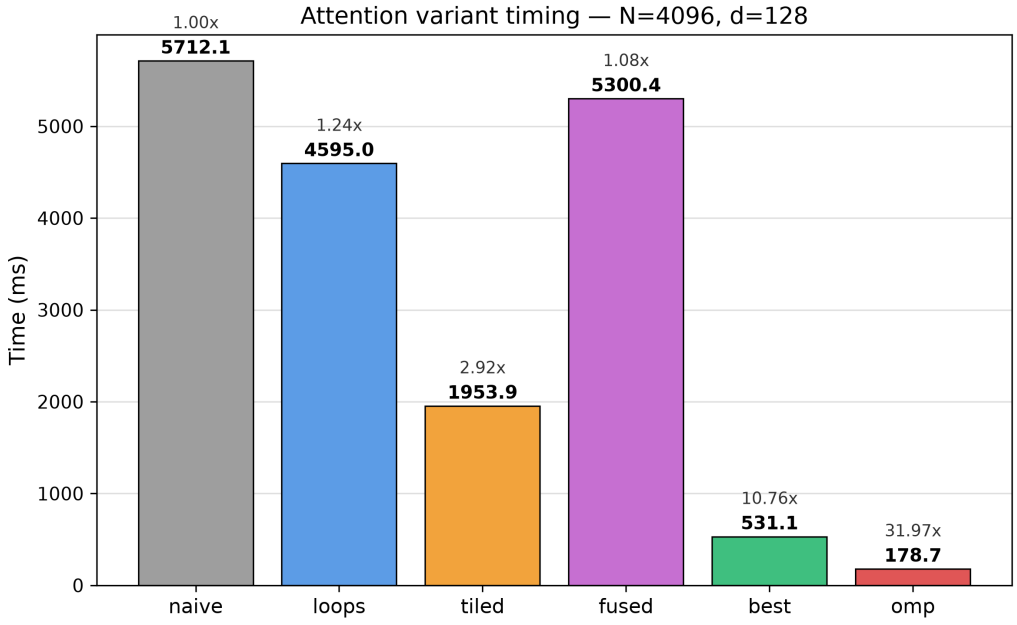


Figure 4: Variant timing, N=4096, d=128.

traffic caps performance near 5 GFLOP/s.

(c) attn_fused: row-wise fusion

Each query row is processed independently: one row of S is computed, softmaxed immediately, and accumulated into O; the $N \times N$ matrix is never stored. In principle this removes $\sim 2/3$ of the S traffic. In practice it merely matches naive ($1.01\times$): every row re-streams the entire K and V matrices (2×1 MiB per row, ≈ 8.6 GB total at N=4096, d=64), which is $32\times$ more traffic than the N^2 stores it saves. Fusion trades an N^2 traffic problem for an $N^2 d$ traffic problem, and the scalar `expf` calls sit on the critical path. Its arithmetic intensity (≈ 0.5 FLOP/byte, see

Q5) confirms this.

(d) attn_best

Combines the winning techniques:

1. **Explicit AVX2+FMA intrinsics** for both matrix products — a full 8-lane `_mm256_fmadd_ps` accumulator for the $QK^\top$ dot products, and broadcast-scale-FMA for $O+ = s \cdot V_j$, with remainder handling for d not divisible by 8. This does not rely on the auto-vectoriser.
2. **Tiling** with T_I=16, T_J=64 so the tile working set stays in L1.
3. **Software prefetch** (`__builtin_prefetch`) of the next K and V tiles to hide L2 latency.
4. **Softmax normalisation** via reciprocal multiply (`x * (1/sum)`) so the pass vectorises trivially.

Result: **8.26× over naive** at N=4096, d=64, reaching 12.9–16.2 GFLOP/s (∼10−13% of the single-core FMA peak). Remaining limits: (i) the 64 MiB S matrix must still be written and read twice; (ii) the dot product has a serial FMA dependency chain on a single accumulator, so the two FMA ports are never saturated — a register-blocked 4×1 microkernel with multiple accumulators would raise ILP; (iii) `expf` remains scalar.

Q4. OpenMP

`attn_omp.c` parallelises the three independent outer loops of `attn_best.c` with `#pragma omp parallel for` (`schedule(dynamic,1)` for the tiled matmuls, `schedule(static)` for softmax). Rows of S and O are thread-private and Q, K, V are read-only shared, so there are no races; `make check` passes with the same errors as `attn_best`.

Threads	Time (ms)	GFLOP/s	Speedup	Parallel efficiency
1	391.6	10.97	1.00×	100%
2	192.9	22.26	2.03×	102%
4	112.4	38.22	3.48×	87%
8	103.8	41.37	3.77×	47%
16	115.6	37.15	3.39×	21%
32	135.9	31.60	2.88×	9%
64	112.4	38.20	3.48×	5%
128	126.1	34.07	3.10×	2%
256	175.5	24.47	2.23×	1%

Table 4: OpenMP scaling, N=4096, d=64. Speedup and efficiency are relative to the 1-thread run.

Scaling analysis.

- **1-thread baseline note:** `attn_omp` with 1 thread (10.97 GFLOP/s) is *slower* than plain `attn_best` (12.86 GFLOP/s) — the OpenMP runtime adds fork/join overhead for the three parallel regions per call plus dynamic-scheduling bookkeeping.
- **1→2 threads (102% efficiency, superlinear):** the working set (64 MiB S + 3 MiB $Q/K/V/O$) exceeds the 8 MiB L3 for a single thread; splitting it across two cores increases effective last-level cache capacity and the hit rate.

- **2→4 threads (87%):** four physical cores do genuine work; shared L3 and DRAM bandwidth begin to be contended.
- **4→8 threads (47%):** only 4 of 8 logical CPUs own FMA/AVX units; the SMT siblings mostly issue loads. Since the kernel already saturates load bandwidth, SMT adds little.
- **>8 threads (negative scaling):** oversubscription adds scheduler and synchronisation overhead and performance *decreases*. Beyond 4 physical cores the machine is thread-supply-bound, not work-bound.
- The plateau at ∼41 GFLOP/s (∼8% of the 518 GFLOP/s all-core peak) marks the transition to memory-bandwidth-bound execution: all threads stream the shared 64 MiB S matrix through DRAM.

Q5. Roofline

Arithmetic intensity

Total FLOPs for attention: $4N^2d = 4.295 \times 10^9$ at N=4096, d=64.

Non-fused variants (naive, loops, tiled, best, omp) touch Q, K, V, O ($4{\cdot}Nd{\cdot}4\,\mathrm{B} \approx 4.2\,\mathrm{MB}$, negligible) plus the score matrix S: written once, read+written once, read once:

$$\text{traffic} \approx 4N^2 \times 4\,\mathrm{B} = 268\,\mathrm{MB} \quad \Rightarrow \quad \mathrm{AI} \approx \frac{4.295 \times 10^9}{268 \times 10^6} \approx 15\text{–}16\ \mathrm{FLOP/byte}$$

Fused variant never stores S, but streams the full K and V matrices once per query row:

$$\text{traffic} \approx 2N(Nd\,4\,\mathrm{B}) = 8.6\,\mathrm{GB} \quad \Rightarrow \quad \mathrm{AI} \approx \frac{4.295 \times 10^9}{8.6 \times 10^9} \approx 0.5\ \mathrm{FLOP/byte}$$

Roofline plot

Interpretation

- **fused** (AI ≈ 0.5) sits far left of the per-core ridge (2.8 FLOP/byte), deep in the bandwidth-limited region, and lands *below* even the ideal bandwidth line because its streamed K/V rows get no reuse: achieved traffic exceeds the compulsory model.
- **naive / loops** (AI ≈ 15) are to the right of the ridge in theory yet reach only 1.6–1.8 GFLOP/s. Their gap is achieved traffic and issue rate, not compulsory bandwidth: strided accesses multiply effective bytes moved, and scalar inner loops cannot feed the FMA ports.
- **tiled** (4.99 GFLOP/s) recovers much of the gap via L1 reuse but still pays full S-matrix traffic with limited ILP.
- **best** (12.86 GFLOP/s, ∼10% of per-core peak) is limited by the serial FMA dependency chain (one accumulator) and the scalar softmax — an instruction-scheduling limit, not bandwidth: at AI 15 it needs only ∼0.9 GB/s.
- **omp (8 threads)** (41.37 GFLOP/s) is the closest point to the ceiling. Its plateau across 8–256 threads shows the *effective* achievable bandwidth with this access pattern is the binding constraint, well below the nominal 46.9 GB/s.

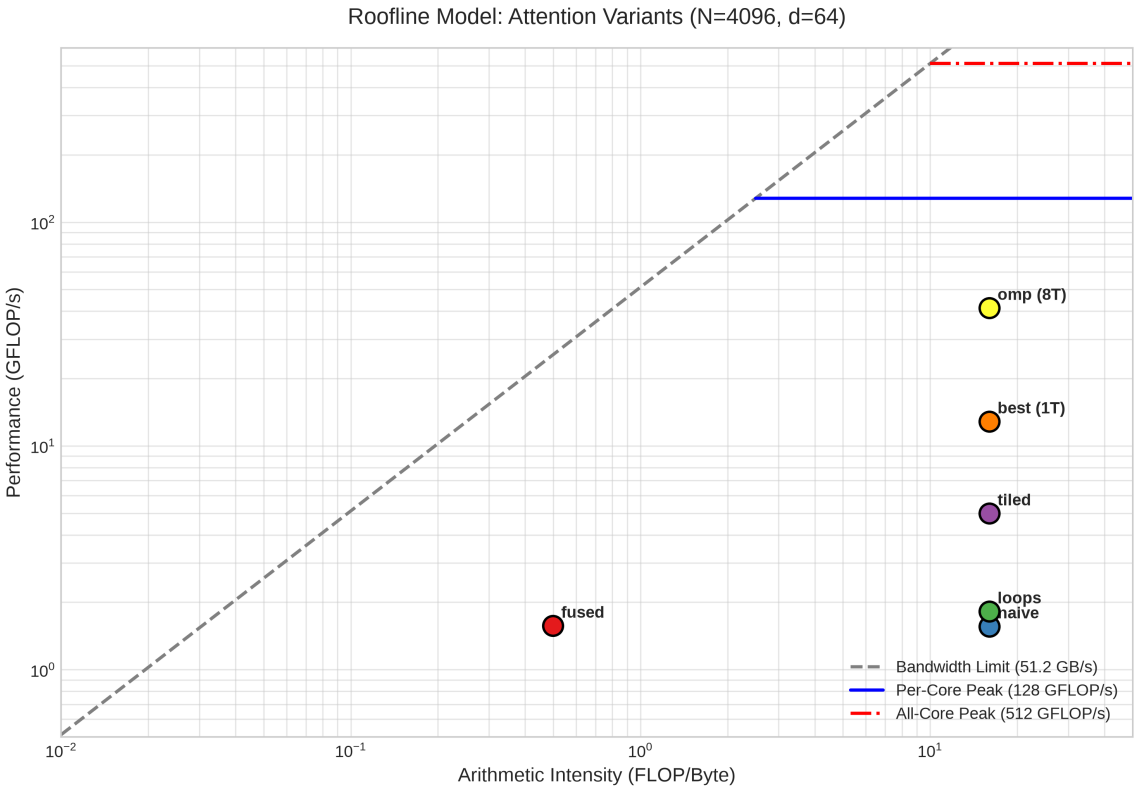


Figure 5: Roofline model for the attention variants at N=4096, d=64. The dashed line is the DDR4 bandwidth limit; the solid/plateau lines are the per-core and all-core FMA peaks. All variants fall below the bandwidth ceiling: none reaches a compute-limited regime.

Summary. The limiting resource progresses across variants from memory latency / poor locality (naive) → L1/L2 locality (tiled) → instruction-level parallelism (best) → memory bandwidth and SMT resource limits (omp). Further single-thread gains would come from multiple FMA accumulators (register blocking), a vectorised `expf`, and keeping the O accumulation resident in registers; further parallel gains require reducing or tiling the N^2 score-matrix traffic itself (flash-attention style online softmax), since bandwidth — not thread count — is the wall.